Reval. | Multipagas.

COI. | Gestión y Monitoreo de Apoyo al Negocio

GERENCIA DE GESTIÓN DEL NEGOCIO · INNOVACIÓN

Guía para analistas

Cómo construir, probar y entregar un módulo de COI con base de datos MySQL, listo para integrarse sin ajustes.

Versión 1.0 · 21 de octubre de 2026

Uso interno · Información confidencial

Contenido

1 Requisitos obligatorios (el .zip se devuelve si no cumple)

2 Ids oficiales

3 Prepare su equipo (una sola vez)

4 Estructura

5 El manifiesto (module.json)

6 Base de datos MySQL

7 El servidor (server/index.js)

8 La interfaz (client/index.jsx)

9 Pruebas (test/*.test.js)

10 Entregue el módulo

11 Qué pasa con su .zip

Cada analista construye su módulo en su propia copia del portal y lo entrega en un **.zip**. Innovación lo revisa con el validador, lo integra con un solo comando y lo publica. Si el .zip cumple esta guía, entra el mismo día sin ajustes; si no, se devuelve con la lista de errores.

> Referencia completa: `modulos/_ejemplo/novedades-turno` (manifiesto, API, tablas MySQL, interfaz, estados vacíos y pruebas). La plantilla con la que arranca su módulo está en `modulos/_plantilla`.

1. Requisitos obligatorios (el .zip se devuelve si no cumple)

#	Requisito	Cómo se comprueba
1	El archivo se llama `<id>-<version>.zip` y tiene **una sola carpeta raíz** `<id>/`	Lo genera `npm run modulo:empaquetar -- <id>`
2	El `id` es el **oficial del plan** (tabla de la sección 2). Un módulo fuera del plan lleva `"nuevo": true` y se aprueba antes	Validador
3	`npm run modulo:validar -- <id>` termina **sin errores**. Cada aviso que quede se explica en el README	Validador
4	`npm test` y `npm run build` terminan sin errores, **con** MySQL de pruebas configurado	Se repite al integrar
5	Base de datos **MySQL**: todas las tablas empiezan por el prefijo del módulo; migraciones numeradas; InnoDB y utf8mb4; sin triggers ni procedimientos	Validador
6	Todo valor en el SQL va como parámetro `?`. Nunca `${…}` ni `+` dentro del SQL	Validador (error)
7	**Cero datos ficticios**: nada de ejemplos, semillas ni cifras inventadas. Sin datos, los contadores muestran 0 y las listas un estado vacío	Revisión
8	Interfaz hecha con `@coi/sdk`, colores y fuentes del portal (variables CSS), clases con el prefijo `<id>-`, usable en celular (390 px)	Validador y revisión
9	Sin credenciales en el código ni archivos `.env`. La configuración propia va por `config.env()`	Validador (error)
10	README con las secciones de la plantilla: Qué hace, Tablas, Endpoints, Roles, Cambios de esta versión	Revisión
11	Pruebas: como mínimo el flujo principal, la validación de datos y el acceso (401 sin sesión; 403 si hay roles)	`npm test`
12	Cada entrega sube la `version` (1.0.0 → 1.0.1 → 1.1.0) y la anota en «Cambios de esta versión»	Importador

2. Ids oficiales

Su módulo toma el lugar de la opción «En construcción» del menú. Use **exactamente** este id y este grupo.

id	Código	Módulo	Grupo del menú	Prefijo sugerido
`validacion`	N1	Validación documental	Operación	`validacion_`
`novedades`	N2	Novedades	Operación	`novedades_`
`alertas`	N4	Alertas	Operación	`alertas_`
`fincomercio`	N8	Fincomercio	Operación	`fincomercio_`
`oficinas`	N7	Oficinas	Operación	`oficinas_`
`reporteria`	N5	Reportería y comités	Cifras	`reporteria_`
`facturacion`	N6	Facturación	Cifras	`facturacion_`
`ptr`	N9	Transacciones en vivo	Cifras	`ptr_`
`malas`	G1	Malas prácticas	Módulos Apps Script	`malas_`
`papeleria`	G2	Papelería	Módulos Apps Script	`papeleria_`
`usuarios`	G3	Usuarios y planta	Módulos Apps Script	`usuarios_`
`cheques`	G4	Cheques y arqueos	Módulos Apps Script	`cheques_`
`agrario`	G5	Banco Agrario	Módulos Apps Script	`agrario_`
`ingresos`	G6	Correo de ingresos	Módulos Apps Script	`ingresos_`

La lista viva está en `client/src/lib/estados.js` (`PENDIENTES`); el validador usa esa.

3. Prepare su equipo (una sola vez)

1. **Node.js 20 o 22** (mínimo 18.18) y una copia actualizada del repositorio de COI.
2. **MySQL 8** local. En Windows, cualquiera de estas opciones:
   - *MySQL Installer* (dev.mysql.com → MySQL Installer for Windows): instale «MySQL Server 8.0» y «MySQL Workbench».
   - *XAMPP*: sirve su MariaDB 10.6 o superior para desarrollar; producción es MySQL 8.
3. Cree la base y el usuario de desarrollo (Workbench o consola `mysql -u root -p`):

```
CREATE DATABASE coi_local CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'coi'@'localhost' IDENTIFIED BY 'clave-local';
GRANT ALL ON coi_local.* TO 'coi'@'localhost';
GRANT ALL ON `coi\_test%`.* TO 'coi'@'localhost';   -- bases temporales de las pruebas
```

4. Cree el archivo `.env` en la raíz del repositorio (nunca se entrega ni se sube a Git):

```
COI_MYSQL_URL=mysql://coi:clave-local@127.0.0.1:3306/coi_local
COI_TEST_MYSQL_URL=mysql://coi:clave-local@127.0.0.1:3306/coi_local
```

5. Instale y cree su módulo desde la plantilla (con el id oficial):

```
npm install
npm run modulo:nuevo -- validacion --autor "Su nombre" --correo su.correo@reval.com.co
npm run modulos:bd                  # comprueba la conexión a MySQL
npm run modulos:migrar -- validacion   # crea las tablas del módulo
npm run dev                         # http://localhost:4000/validacion (ingreso de
demostración)
```

Comando	Para qué
`npm run modulo:nuevo -- <id> [--prefijo abc_] [--sin-bd]`	Crea el módulo desde la plantilla (nombre, grupo e icono salen del plan)
`npm run modulos:migrar [-- <id>]`	Aplica las migraciones pendientes
`npm run modulos:bd`	Diagnóstico: conexión, versión, tablas y migraciones pendientes
`npm run modulo:validar -- <id>`	Revisa el módulo contra esta guía
`npm run modulo:empaquetar -- <id>`	Genera `<id>-<version>.zip` para entregar
`npm run modulos:listar`	Módulos instalados en su copia

4. Estructura

Mientras desarrolla, el módulo vive en dos carpetas (el portal se publica como dos servicios); el .zip las une así:

```
validacion/
├── module.json                manifiesto
├── README.md                  Qué hace · Tablas · Endpoints · Roles · Cambios de esta versión
├── server/                    → server/src/modulos/validacion/server/
│   ├── index.js               API del módulo
│   └── migraciones/
│       ├── 001_inicial.sql    tablas iniciales
│       └── 002_estado.sql     cada cambio posterior, en un archivo nuevo
├── client/                    → client/src/modulos/validacion/
│   ├── index.jsx              pantalla principal
│   └── validacion.css         estilos (clases .validacion-…)
└── test/
    └── api.test.js            pruebas (node:test)
```

No se aceptan: `node_modules/`, `package.json`, archivos ocultos (`.env`, `.git`), carpetas fuera de `server/`, `client/`, `test/`, ni archivos de más de 2 MB (10 MB en total).

5. El manifiesto (`module.json`)

```
{
  "id": "validacion",
  "nombre": "Validación documental",
  "version": "1.0.0",
  "descripcion": "Recepción y validación de los soportes documentales de cada cliente.",
  "autor": { "nombre": "Ana Analista", "correo": "ana.analista@reval.com.co" },
  "menu": { "grupo": "Operación", "etiqueta": "Validación documental", "icono": "filecheck",
"orden": 10 },
  "acceso": { "roles": [] },
  "db": { "motor": "mysql", "prefijo": "validacion_" },
  "dependencias": { "server": {}, "client": {} }
}
```

Campo	Regla
`id`	Id oficial (sección 2). Minúsculas, números y guiones

Campo	Regla
`version`	Semver `MAYOR.MENOR.PARCHE`; sube en cada entrega
`menu.grupo`	El del plan. `icono`: uno de los del portal (`filecheck`, `inbox`, `bell`, `chart`, `wallet`, …)
`acceso.roles`	`[]` = todos los usuarios con sesión. Con roles (p. ej. `["lider"]`) solo ellos y `admin` lo ven
`db.motor`	`"mysql"`, o `"ninguna"` si el módulo no guarda datos
`db.prefijo`	2 a 11 minúsculas o números terminando en `_`; único en el portal
`dependencias`	Solo paquetes aprobados (hoy `dayjs` y `zod`). Pida cualquier otro **antes** de usarlo

6. Base de datos MySQL

Todos los módulos comparten la base MySQL del portal (`COI_MYSQL_URL`); cada uno es dueño solo de sus tablas, identificadas por su prefijo.

Regla	Ejemplo
Toda tabla, índice de tabla, vista o llave foránea apunta a tablas `prefijo_…`	`validacion_soporte`
Nombres en minúsculas y `snake_case`	`validacion_soporte_estado`
`ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci` en cada `CREATE TABLE`	—
Llave `id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY`	—
Fechas `DATETIME` / `DATE`; dinero `DECIMAL(16,2)` (nunca `FLOAT`)	`valor DECIMAL(16,2) NOT NULL`
Índices para los filtros y ordenamientos que usa la pantalla	`INDEX validacion_soporte_estado (estado, creado_en)`
Migraciones `NNN_descripcion.sql` (001, 002, …). **Nunca edite una ya entregada**: cree la siguiente	`002_agrega_observacion.sql`
Prohibido: `CREATE/DROP DATABASE`, `USE`, `GRANT`, usuarios, `SET GLOBAL`, triggers, procedimientos, funciones, eventos, `LOAD DATA`, `START TRANSACTION`, tocar `coi_migraciones`	—
Sin `INSERT` de datos de ejemplo. Solo catálogos reales y fijos (p. ej. tipos de documento)	—

```
-- server/migraciones/001_inicial.sql
CREATE TABLE validacion_soporte (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  cliente VARCHAR(80) NOT NULL,
  tipo VARCHAR(40) NOT NULL,
  estado VARCHAR(12) NOT NULL DEFAULT 'pendiente',
  valor DECIMAL(16,2) NULL,
  creado_por VARCHAR(120) NOT NULL,
  creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX validacion_soporte_estado (estado, creado_en)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Las migraciones se aplican solas al primer uso del módulo (y con `npm run modulos:migrar`), en orden, y quedan registradas en `coi_migraciones`. En MySQL los `CREATE/ALTER` se confirman solos: haga una migración pequeña por cambio. La zona horaria de la sesión es Colombia (`-05:00`); los `DECIMAL` llegan como texto (conviértalos con `Number()` si los va a sumar en JavaScript).

7. El servidor (`server/index.js`)

```
const fallo = (status, mensaje) => Object.assign(new Error(mensaje), { status });

export default async function registrar({ router, db, config, log, acceso }) {
  router.get('/soportes', async (req, res) => {
    const estado = String(req.query.estado || 'pendiente');
    const { rows } = await db.query(
      'SELECT id, cliente, tipo, estado, creado_en FROM validacion_soporte WHERE estado = ?
ORDER BY creado_en DESC LIMIT 200',
      [estado]);
    res.json(rows);
  });

  router.post('/soportes', async (req, res) => {
    const cliente = String(req.body.cliente || '').trim();
    if (cliente.length < 2) throw fallo(400, 'Seleccione el cliente.');
    const { info } = await db.query(
      'INSERT INTO validacion_soporte (cliente, tipo, creado_por) VALUES (?, ?, ?)', [cliente,
req.body.tipo, req.usuario.nombre]);
    res.status(201).json({ id: info.insertId });
  });
}
```

Recibe	Qué es
`router`	Router de Express montado en `/api/m/<id>`. Ya exige sesión y los roles de `acceso.roles`
`req.usuario`	`{ id, nombre, correo, cargo, roles }` de quien llama
`db`	`db.query(sql, [valores])` → `{ rows, info }` (`info.insertId`, `info.affectedRows`); `db.transaccion(async (tx) => …)`
`acceso`	`acceso.puedeAcceder(req.usuario, ['lider'])` para permisos finos dentro del módulo
`config`	`config.env('TOKEN')` lee la variable `MOD_<ID>_TOKEN` (credenciales de servicios externos)
`log`	`log.info / warn / error` con el nombre del módulo

Reglas: valide **toda** entrada (tipos, largos, listas permitidas); responda errores con `status` 4xx y un mensaje en español claro para el usuario (los 5xx se muestran como «error interno»); use `async` y deje que el portal capture las excepciones; varias escrituras relacionadas van en `db.transaccion`. No use `process.env`, `child_process`, `eval`, `process.exit` ni escriba en disco.

8. La interfaz (`client/index.jsx`)

```
import { Card, ErrorModulo, Kpis, Loading, PageHead, Table, Vacio, useRecurso } from '@coi/sdk';
import './validacion.css';

export default function Modulo({ sub, usuario, api }) {
  const soportes = useRecurso(api, '/soportes');          // GET /api/m/validacion/soportes
  if (soportes.error) return <ErrorModulo error={soportes.error} reintentar={soportes.reload}
/>;
  if (!soportes.data) return <Loading />;
  const lista = soportes.data;
  return (
    <>
      <PageHead title="Validación documental" sub="Soportes recibidos y su validación por
cliente." />
      <Kpis items={[['Pendientes', lista.length, 'Por validar', 'filecheck']]} />
      <Card title="Soportes" flush>
        {lista.length ? <Table headers={['Cliente', 'Tipo']} rows={lista.map((s) => [s.cliente,
s.tipo])} />
          : <Vacio titulo="No hay soportes pendientes" texto="Los soportes aparecerán aquí al
radicarse." />}
      </Card>
    </>
  );
}
```

El portal entrega `usuario`, `api` (`get`, `post`, `put`, `patch`, `del`, todas relativas a `/api/m/<id>`) y `sub` (segundo tramo de la ruta, `/validacion/<sub>`, para subpantallas). Solo puede importar `react`, `react-router-dom` y `@coi/sdk`:

Grupo	Elementos de `@coi/sdk`
Diseño	`PageHead`, `Kpi`, `Kpis`, `Card`, `Table`, `Cell`, `Tabs`, `Tag`, `Bar`, `HBars`, `LineChart`, `Spark`, `ListRow`, `DocRow`, `Field`, `fieldStyle`, `areaStyle`, `Icon`, `CountUp`
Estados	`Loading` / `Cargando`, `Vacio` (sin datos), `ErrorModulo` (error; un 503 explica que la base está pendiente de configurar)
Interacción	`useOverlay()` → `toast`, `modal`, `closeModal`; `useApp()` → `go('ruta')`
Sesión y datos	`useAuth()`, `puedeAcceder(usuario, roles)`, `crearApi`, `useRecurso(api, ruta)`, `CLIENTES`, `SEGMENTOS`
Formato	`fmt`, `pct`, `money`, `longDate`, `shortDate`, `cap`, `norm`, `hhmm`

Marca y diseño:

- Colores solo con variables: `var(--navy)` (#242A54), `var(--green)` (#048C38), `var(--cyan)` (#21BBEA), `var(--gold)` (#F6CF46, **solo acento**), `var(--line)`, `var(--muted)`, `var(--red)`, `var(--surface)`. No escriba colores en hexadecimal.
- Fuentes del portal (Ubuntu / Noto Sans) con `var(--font-display)` y `var(--font-body)`; no cargue otras ni logos.
- Toda clase CSS empieza por el id: `.validacion-tabla`, `.validacion-form .btn`. Nada de selectores globales (`td`, `.card`, `:root`).
- Textos en español, con tildes, en tono institucional. Fechas y cifras en formato colombiano (`fmt`, `money`).
- Revise la pantalla en 390 px de ancho: sin desplazamiento horizontal.
- Sin datos: KPIs en 0 y `Vacio` con una indicación útil. Nunca rellene con ejemplos.

9. Pruebas (`test/*.test.js`)

```
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { probarModulo, requiereMySQL } from '#coi/pruebas';

let m;
before(async () => { m = await probarModulo(new URL('..', import.meta.url)); });
after(() => m.cerrar());

test('crea y lista soportes', { skip: requiereMySQL }, async () => {
  const c = await m.pedir('POST', '/soportes', { cliente: 'BBVA', tipo: 'planilla' });
  assert.equal(c.status, 201);
  assert.equal((await m.pedir('GET', '/soportes')).body.length, 1);
});
test('valida los datos', { skip: requiereMySQL }, async () => {
  assert.equal((await m.pedir('POST', '/soportes', {})).status, 400);
});
test('exige sesión', async () => {
  assert.equal((await m.pedir('GET', '/soportes', undefined, { como: 'anonimo' })).status, 401);
});
```

`probarModulo` crea una base MySQL temporal (`coi_test_…`) con las migraciones aplicadas y la borra al terminar. `m.pedir(método, ruta, cuerpo, { como })` llama al API; `como` simula otro usuario (`{ ...USUARIO_PRUEBA, roles: ['lider'] }`) o `'anonimo'`. Sin `COI_TEST_MYSQL_URL` las pruebas con `{ skip: requiereMySQL }` se omiten; **antes de entregar córralas con MySQL**.

10. Entregue el módulo

```
npm run modulo:validar -- validacion     # 0 errores
npm test                                 # con COI_TEST_MYSQL_URL en .env
npm run build
npm run modulo:empaquetar -- validacion  # crea validacion-1.0.0.zip en la raíz del repositorio
```

Envíe el .zip a Innovación. **Lista de chequeo:**

- [ ] Id y grupo oficiales; `version` mayor que la entrega anterior.
- [ ] `modulo:validar` sin errores; avisos explicados en el README.
- [ ] `npm test` (con MySQL) y `npm run build` sin errores.
- [ ] Tablas con el prefijo; migraciones nuevas para cada cambio; ninguna migración entregada fue editada.
- [ ] SQL solo con `?`; entradas validadas; errores en español.
- [ ] Sin datos ficticios; contadores en 0 y estados vacíos probados con la base vacía.
- [ ] Probado en 1440 px y 390 px; colores, fuentes y clases según la marca.
- [ ] README con Qué hace, Tablas, Endpoints, Roles y Cambios de esta versión.

11. Qué pasa con su .zip

1. Innovación ejecuta `npm run modulo:importar -- validacion-1.0.0.zip` (con `--reemplazar` si es una versión nueva; la anterior queda respaldada).
2. Corre `npm test`, `npm run build` y revisa la pantalla. Si todo pasa, se integra y se publica: la opción deja de mostrar «En construcción».
3. Si algo falla, recibe de vuelta la salida del validador y las observaciones. Errores frecuentes:

Mensaje	Solución
`toda tabla del módulo debe empezar por su prefijo`	Renombre la tabla en una migración **nueva** o, si aún no se ha entregado, en la original
`SQL armado con ${…}`	Cambie el valor por `?` y páselo en el arreglo de parámetros
`sintaxis de PostgreSQL`	`SERIAL` → `INT AUTO_INCREMENT`, `TIMESTAMPTZ` → `DATETIME`, `ILIKE` → `LIKE`
`importación no permitida`	Use `@coi/sdk` o pida aprobar la dependencia
`parece contener un secreto`	Pase la credencial a `config.env('NOMBRE')` (variable `MOD_<ID>_NOMBRE`)

Mensaje	Solución
`ya está instalado (versión …)`	Suba la `version` en `module.json`
`colores escritos a mano` / `selectores no empiezan por …`	Use variables del portal y clases `.<id>-…`